```
    public override void OnActionExecuting(                    ❷
        ActionExecutingContext context)                        ❷
    {
        if (!context.ModelState.IsValid)                       ❸
        {
            context.Result =                                   ❹
                new BadRequestObjectResult(context.ModelState); ❹
        }
    }
}
```

❶ For convenience, you derive from the ActionFilterAttribute base class.

❷ Overrides the Executing method to run the filter before the Action executes

❸ Model binding and validation have already run at this point, so you can check the state.

❹ If the model isn't valid, sets the Result property, which short-circuits the action execution

This attribute is self-explanatory and follows a similar pattern to the resource filter in section 22.1.2, but with a few interesting points:

- I have derived from the abstract `ActionFilterAttribute`. This class implements `IActionFilter` and `IResultFilter`, as well as their async counterparts, so you can override the methods you need as appropriate. This prevents needing to add an unused `OnActionExecuted()` method, but using the base class is entirely optional and a matter of preference.
- Action filters run after model binding has taken place, so `context.ModelState` contains the validation errors if validation failed.
- Setting the `Result` property on `context` short-circuits the pipeline. But due to the position of the action filter stage, only the action method execution and later action filters are bypassed; all the other stages of the pipeline run as though the action executed as normal.